Impact Analysis: Renaming Database Tables (Adding Prefixes)

This document analyzes the specific consequences of renaming a core database table in the TalentOps application (e.g., changing `employee_finance` to `tops_employee_finance`) and provides a checklist for resolving the resulting system-wide breaks.

🚨 The Immediate Consequences

Adding a prefix to a core table (or all tables) without warning will instantly cause **massive, system-wide failures**. Based on a live scan of the local codebase, here is exactly what will break if `employee_finance` is renamed to `tops_employee_finance`:

1. Frontend React/Next.js API Calls Will Fail

The frontend code is hardcoded to talk to Supabase using the exact current string `'employee_finance'`. The moment you rename the table, the database will return a `Relation "employee_finance" does not exist` error, crashing these UI components:

- `components/shared/AddEmployeeModal.jsx` (Lines 179-189): **Adding a new employee will fail.**
- `components/shared/EditEmployeeModal.tsx` (Lines 216-419): **Editing employee salaries will fail.**
- `components/shared/payslip/PayslipFormModal.jsx` (Line 346): **Generating payslips will break.**
- `components/executive/pages/ModulePage.jsx` (Lines 619, 1082): **The Executive dashboard will fail to load financial stats.**
- `components/manager/pages/ModulePage.jsx` (Lines 1100, 1128): **The Manager dashboard financial views will fail to load.**

2. Live Supabase Backend & Payroll Logic Will Break

Active SQL logic and internal calculation utilities expect the exact `employee_finance` name. When renamed, these will crash:

- `utils/payrollCalculations.js`: **Payroll batch calculations will entirely stop working.**
- `rpc_get_my_profile_v2.sql` & `rpc_get_my_profile_v3.sql`: **Users logging in will get empty profiles or crash** because the `LEFT JOIN` used to fetch their salary data will fail looking for the old table name.

3. Database Views & Functions Will Corrupt

- **Views:** Any View (like `view_employee_aggregate_performance`) that implicitly relies on `employee_finance` data will break because its underlying saved SQL `SELECT` query is pointing to a dead table name.
- **Edge Functions:** Deno functions (`supabase/functions/`) will throw 500 errors if they attempt to write context data into or read from a renamed table.

🛠️ The Resolution Guide (How to Safely Rename a Table)

If you have a strict business requirement to add the `tops_` prefix to your tables, **you must execute the following checklist in this exact order to prevent downtime:**

Step 1: Update the Frontend API Calls

You must find and replace all hardcoded Supabase client calls in your frontend codebase.

- **Action:** Use global search (Ctrl+Shift+F) for `.from('employee_finance')` and `.from("employee_finance")`.
- **Fix:** Change every instance across your `components/` and `utils/` folders to `.from('tops_employee_finance')`.

Step 2: Update Local SQL Files & RPCs

You must update the raw SQL queries that execute on the server side.

- **Action:** Search your `.sql` files for `public.employee_finance` or `employee_finance`.
- **Fix:** Specifically rewrite the `JOIN` clauses in `rpc_get_my_profile_v3.sql`, `rpc_get_my_profile_v2.sql`, and `rpc_generate_monthly_payroll.sql` to reference the new `tops_employee_finance` table name.

Step 3: Deploy the Updated SQL Functions to Supabase

Updating the `.sql` text files locally does nothing on its own.

- **Action:** You must physically copy the updated `CREATE OR REPLACE FUNCTION` scripts and run them directly inside the Supabase SQL Editor. This overwrites the old server-side logic with the new prefixed table names.

Step 4: Rename the Physical Table in Supabase

Only *after* the new logic is pushed should you alter the actual database schema.

- **Action:** Open the Supabase Table Editor UI, select `employee_finance`, and alter its name to `tops_employee_finance`.

Step 5: Deploy the Frontend

- **Action:** Push your updated React/Next.js frontend code (from Step 1) to production (e.g., Vercel, Netlify). The live application will now seamlessly make API calls to the newly prefixed tables using the updated RPCs.

Strongly Recommended Alternative

Do not add a prefix to all your tables unless absolutely necessary.

Adding prefixes (like `wp_posts` in WordPress) is a legacy practice from when multiple different applications had to share a single, monolithic database connection to prevent naming collisions.

With modern tools like Supabase, your TalentOps app has its own **dedicated, isolated Postgres database**. Table naming collisions are impossible because no other external app lives inside this specific Supabase project environment. Renaming 69 tables will require dozens of hours of painstaking refactoring (updating hundreds of `.from()` calls, rewriting all 6 Views, and updating 44 SQL functions) just to keep the app working exactly as it already does today.